

Software Requirements Specification (SRS)

Voice of Northern

Digital Platform for Student Rights, Community Engagement, and Organizational Management

Document Information

Item	Details
Project Name	Voice of Northern
Document Type	Software Requirements Specification (SRS)
Project Type	Real Production Grade Web Platform
Development Model	Agile / Iterative Development
Frontend	Next.js
Backend	Node.js + Express.js
Database	PostgreSQL
Authentication	JWT
Deployment	VPS-Based Deployment
UI Style	Modern, Professional, Student-Friendly, Dark Mode First

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of the “Voice of Northern” platform.

The purpose of the system is to build a centralized digital platform for a university student organization that focuses on:

- Student rights advocacy

- Complaint management
- Community engagement
- Event management
- Content publishing
- Membership management
- Organizational communication

The document is intended for:

- Developers
- Designers
- Project Managers
- Organization Administrators
- Stakeholders
- Future Contributors

This SRS combines both IEEE-style software requirement documentation and practical production-oriented product planning.

1.2 Scope

Voice of Northern is a web-based platform designed for a real student organization.

The platform will replace fragmented communication systems such as:

- Facebook groups
- Messenger groups
- WhatsApp groups
- Manual complaint handling
- Manual member management

The platform aims to provide:

- Structured complaint handling
- Public awareness system
- Event registration system
- Verified membership system
- Organizational dashboard
- Role-based administration
- Secure communication workflow

The system is designed as a scalable production-grade application.

1.3 Goals and Objectives

The main goals of the platform are:

1. Centralize organizational operations
 2. Digitize student complaint management
 3. Improve transparency and accountability
 4. Improve communication efficiency
 5. Build a trusted student support ecosystem
 6. Manage events and student activities
 7. Maintain verified membership records
 8. Build a scalable long-term community platform
-

1.4 Intended Audience

This document is intended for:

- Software Engineers
 - Frontend Developers
 - Backend Developers
 - Database Engineers
 - DevOps Engineers
 - UI/UX Designers
 - Organization Administrators
 - QA/Test Engineers
 - Stakeholders
-

1.5 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token
UI	User Interface
UX	User Experience
API	Application Programming Interface
Complaint Officer	Organization member responsible for complaint review
Event Manager	User role responsible for events

Term	Meaning
Content Writer	User role responsible for content publishing
Soft Delete	Marking records deleted without permanent removal

2. Overall Description

2.1 Product Perspective

Voice of Northern is a centralized community platform.

The system consists of:

- Public Website
- Complaint Management System
- Membership Verification System
- Event Registration System
- Content Management System
- Notification System
- Administrative Dashboard

The platform will be accessible through modern web browsers and optimized for:

- Desktop
- Tablet
- Mobile Devices

The application follows a modular architecture to support future scalability.

2.2 Product Functions

Major functions of the system include:

Public Features

- Browse notices
- Read blogs and magazines
- View events
- View public complaints
- Apply for membership
- Register for events

Complaint Management

- Submit complaint
- Submit anonymous complaint
- Upload evidence
- Track complaint status
- Admin review workflow
- Public publishing after approval

Membership Management

- Open registration
- Student verification
- Manual approval workflow
- Role assignment

Event Management

- Create events
- Manage registrations
- Free/open/paid events
- Contribution support

Content System

- Rich text editor
- Markdown editor
- PDF upload
- Image upload
- Categories and tags

Administrative Features

- Dashboard analytics
- User management
- Complaint moderation
- Notification management
- Audit logs

2.3 User Classes and Characteristics

Guest User

Capabilities:

- Browse public content
- View public complaints

- View events
- Read notices
- Apply for membership

Member

Capabilities:

- Submit complaints
- Register for events
- Access member content
- Receive notifications

Moderator

Capabilities:

- Moderate content
- Verify members
- Manage notices
- Review complaints

Complaint Officer

Capabilities:

- Review complaints
- Update complaint status
- Approve/reject complaints
- Manage complaint workflow

Event Manager

Capabilities:

- Create/manage events
- Approve registrations
- Monitor participation

Content Writer

Capabilities:

- Publish blogs
- Publish magazine content
- Manage categories/tags

Admin

Capabilities:

- Full operational control
- Role assignment
- System management
- Analytics monitoring

Super Admin

Capabilities:

- Full unrestricted access
 - Infrastructure/system control
 - Permission management
 - Critical administrative actions
-

2.4 Operating Environment

Frontend

- Next.js
- React Ecosystem
- Tailwind CSS
- Responsive UI
- TanStack Query

Backend

- Node.js
- Express.js
- REST API Architecture
- Typescript

Database

- PostgreSQL
- ORM: Prisma ORM

Validation

Zod / Joi

Deployment

- VPS-Based Linux Server

- Nginx Reverse Proxy
- PM2 Process Management

Authentication

- JWT-based Authentication

Storage

- Local VPS Storage / Cloud Storage

Logging

Winstone

2.5 Design and Implementation Constraints

- Must support responsive design
 - Must support dark mode
 - Must support RBAC
 - Must protect anonymous users
 - Must support soft delete
 - Must maintain audit logs
 - Must comply with privacy standards
 - Must support future scalability
-

2.6 Assumptions and Dependencies

Assumptions:

- Students have internet access
- Organization moderators verify users manually
- Users provide valid student information
- VPS infrastructure remains available

Dependencies:

- Email service
 - WhatsApp notification integration
 - PostgreSQL database availability
 - File storage system
-

3. System Features and Functional Requirements

3.1 Authentication and Authorization

Description

The system shall provide secure authentication and authorization.

Functional Requirements

FR-AUTH-01

Users shall be able to register.

FR-AUTH-02

Users shall log in using email and password.

FR-AUTH-03

JWT tokens shall be used for authentication.

FR-AUTH-04

The system shall support role-based access control.

FR-AUTH-05

The system shall support password reset.

FR-AUTH-06

The system shall support secure logout.

FR-AUTH-07

Protected routes shall require authentication.

3.2 Membership Management

Description

The system shall support student membership registration and manual verification.

Functional Requirements

FR-MEM-01

Users shall apply for membership.

FR-MEM-02

Users shall provide:

- Department
- Student ID
- Personal information

FR-MEM-03

Admins/moderators shall manually verify membership.

FR-MEM-04

Membership can be approved or rejected.

FR-MEM-05

Verified members shall receive membership status.

FR-MEM-06

The system shall maintain membership records.

FR-MEM-07

Soft delete shall be supported.

3.3 Complaint Management System

Description

The complaint system is the core feature of the platform.

Students can submit complaints regarding injustice, problems, or student rights issues.

Complaints can optionally be anonymous.

Functional Requirements

FR-CMP-01

Users shall submit complaints.

FR-CMP-02

Complaints may be submitted anonymously.

FR-CMP-03

The system shall allow evidence uploads.

Supported file types:

- Image
- PDF
- Video

FR-CMP-04

Complaints shall initially remain private.

FR-CMP-05

Complaint officers/admins shall review complaints.

FR-CMP-06

Approved complaints may become publicly visible.

FR-CMP-07

Anonymous complaints shall remain anonymous publicly.

FR-CMP-08

Only authorized administrators may view original anonymous user information.

FR-CMP-09

Users shall track complaint status.

FR-CMP-10

Complaint statuses:

- Pending
- Under Review
- Approved
- Rejected
- Resolved
- Escalated

FR-CMP-11

The system shall maintain complaint history logs.

FR-CMP-12

Complaint data shall support encryption.

FR-CMP-13

Soft delete shall be supported.

3.4 Event Management System

Description

The platform shall support organizational event management.

Event Types

- Free Events
 - Open Contribution Events
 - Paid Events
-

Functional Requirements

FR-EVT-01

Event managers/admins shall create events.

FR-EVT-02

Users shall register for events.

FR-EVT-03

Paid event registration shall require:

- Payment method
- Transaction ID

FR-EVT-04

Payment screenshot upload shall be optional.

FR-EVT-05

The system shall store event registration records.

FR-EVT-06

Event registration status shall be manageable.

FR-EVT-07

The system shall support event categories.

FR-EVT-08

The system shall support event images and banners.

FR-EVT-09

The system shall support contribution-based participation.

3.5 Notice and Announcement System

Functional Requirements

FR-NOT-01

Admins/moderators shall publish notices.

FR-NOT-02

Notices shall support categories.

FR-NOT-03

Priority levels shall be supported.

FR-NOT-04

Users shall browse notice archives.

FR-NOT-05

Urgent notices shall be highlighted.

3.6 Content Management System

Description

The platform shall support blogs, magazines, and organizational content.

Functional Requirements

FR-CMS-01

Content writers shall create content.

FR-CMS-02

The editor shall support:

- Rich Text Editor
- Markdown Editor

FR-CMS-03

Users shall switch between editors.

FR-CMS-04

The system shall support:

- PDF Upload
- Image Upload

FR-CMS-05

Posts shall support:

- Categories
- Tags

FR-CMS-06

The system shall support drafts.

FR-CMS-07

The system shall support publishing workflow.

FR-CMS-08

Soft delete shall be supported.

3.7 Notification System

Description

The system shall provide multiple notification channels.

Functional Requirements

FR-NTF-01

The system shall support email notifications.

FR-NTF-02

The system shall support in-app notifications.

FR-NTF-03

The system shall support WhatsApp notifications.

FR-NTF-04

Users shall receive complaint updates.

FR-NTF-05

Users shall receive event notifications.

FR-NTF-06

Admins shall broadcast announcements.

3.8 Administrative Dashboard

Description

The system shall provide a production-grade administrative dashboard.

Dashboard Analytics

The dashboard shall display:

- Total complaints
- Resolved complaints
- Pending complaints
- Total members
- Active members
- New members
- Monthly growth analytics

- Yearly growth analytics
 - Total events
 - Active events
 - Event registration analytics
 - Content statistics
 - Recent activity logs
 - Notification metrics
 - Recent complaints
 - Recent posts
 - System activity summary
-

Functional Requirements

FR-ADM-01

Admins shall manage users.

FR-ADM-02

Admins shall assign roles.

FR-ADM-03

Admins shall manage complaints.

FR-ADM-04

Admins shall manage events.

FR-ADM-05

Admins shall manage notices.

FR-ADM-06

Admins shall manage content.

FR-ADM-07

Admins shall access audit logs.

FR-ADM-08

The system shall support activity tracking.

FR-ADM-09

Soft delete recovery shall be supported.

4. External Interface Requirements

4.1 User Interface Requirements

Design Requirements

The UI shall:

- Be modern and professional
- Support dark mode
- Be student-friendly
- Maintain responsive layouts
- Support accessibility standards

Responsive Requirements

Supported layouts:

- Desktop
- Laptop
- Tablet
- Mobile

The system shall:

- Avoid horizontal scrolling
 - Use touch-friendly elements
 - Maintain proper spacing
 - Maintain readable typography
-

4.2 Hardware Interface Requirements

Server Requirements:

- Linux VPS
 - SSD Storage
 - Stable Internet Connection
-

4.3 Software Interface Requirements

Backend Services

- PostgreSQL
- Email Service
- WhatsApp API Integration

Browser Support

- Chrome
 - Firefox
 - Edge
 - Safari
-

5. Non-Functional Requirements

5.1 Performance Requirements

The system shall:

- Load pages efficiently
- Handle concurrent users
- Support optimized database queries
- Minimize API response time
- Optimize media delivery

Expected response time:

- Typical requests under 2 seconds
-

5.2 Security Requirements

Authentication Security

- JWT authentication
- Secure password hashing
- Protected routes

Authorization Security

- RBAC implementation

- Permission-based access

Data Security

- Complaint encryption
- Secure file uploads
- Secure storage

System Security

- Rate limiting
- Input validation
- XSS protection
- CSRF protection
- SQL injection prevention

Monitoring

- Audit logs
- Activity tracking
- Suspicious activity detection

Data Protection

- Soft delete
 - Backup strategy
 - Recovery strategy
-

5.3 Reliability Requirements

The system shall:

- Provide stable uptime
 - Handle unexpected failures
 - Prevent data corruption
 - Support recovery workflows
-

5.4 Scalability Requirements

The system architecture shall support:

- Increased users

- Increased complaints
 - Increased events
 - Additional future modules
 - Mobile application integration
-

5.5 Maintainability Requirements

The system shall:

- Follow modular architecture
 - Maintain clean code structure
 - Use reusable components
 - Maintain documentation
-

5.6 Usability Requirements

The system shall:

- Be easy to navigate
 - Minimize learning curve
 - Support mobile usability
 - Maintain consistent UI patterns
-

6. System Models

6.1 Use Case Overview

Primary Use Cases:

- User Registration
- Login
- Membership Application
- Complaint Submission
- Complaint Review
- Complaint Approval
- Event Registration
- Content Publishing
- Notice Publishing
- Role Management
- Notification Broadcasting

6.2 Data Model Overview

Main Entities:

- Users
- Roles
- Complaints
- Complaint Logs
- Events
- Event Registrations
- Notices
- Blogs
- Categories
- Tags
- Notifications
- Audit Logs

6.3 Complaint Workflow

Submit Complaint → Review → Approve/Reject → Public Visibility (Optional) → Resolution → Archive

6.4 Membership Workflow

Apply → Verification → Manual Review → Approve/Reject → Member Access

6.5 Event Workflow

Create Event → Publish → Registration → Payment Verification (If Paid) → Registration Approval → Event Completion

7. Database Considerations

Main Database Tables

- users
- roles

- permissions
 - memberships
 - complaints
 - complaint_logs
 - events
 - event_registrations
 - notices
 - posts
 - categories
 - tags
 - notifications
 - audit_logs
 - activities
-

8. API Considerations

The backend shall expose RESTful APIs.

Main API modules:

- Authentication API
 - User API
 - Membership API
 - Complaint API
 - Event API
 - Notification API
 - Content API
 - Dashboard API
-

9. Deployment Architecture

Production Deployment

Frontend

- Next.js Application

Backend

- Node.js + Express.js API Server

Database

- PostgreSQL Server

Reverse Proxy

- Nginx

Process Management

- PM2

Hosting

- Linux VPS
-

10. Future Scope

Planned future improvements:

- Mobile Application
 - AI Complaint Classification
 - AI Moderation
 - Petition/Voting System
 - Live Chat System
 - University System Integration
 - Event Participation Tracking
 - Quiz/Competition Module
 - Automatic Student Identity Verification
 - Advanced Analytics
-

11. Risks and Challenges

Potential challenges:

- Anonymous complaint misuse
 - Manual verification workload
 - Media storage growth
 - Notification delivery reliability
 - User privacy concerns
 - Scaling moderation processes
-

12. Conclusion

Voice of Northern is designed as a scalable, secure, and production-grade digital ecosystem for student rights advocacy and community management.

The platform aims to centralize organizational operations, improve transparency, empower student voices, and provide a sustainable digital infrastructure for long-term organizational growth.

The system architecture, requirements, and workflows are designed to support both current operational needs and future scalability.